

Three Way Partition of an Array

Problem Statement

Given an array `arr[]` and two integers `a` and `b`, partition the array into three parts such that:

1. All elements smaller than `a` appear first.
2. All elements in the range `[a, b]` appear next.
3. All elements greater than `b` appear at the end.

The relative order of elements within each group does not need to be maintained.

Example

Input

```
arr = [1, 4, 3, 2, 5, 7, 6, 8]
a = 3
b = 6
```

Output

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

Explanation

The three groups are:

```
Elements < 3:
[1, 2]
```

```
Elements between 3 and 6:
[3, 4, 5, 6]
```

```
Elements > 6:
[7, 8]
```

Therefore:

```
[1, 2] + [3, 4, 5, 6] + [7, 8]  
  
= [1, 2, 3, 4, 5, 6, 7, 8]
```

Approach

This solution uses an **extra temporary array**.

We divide the elements into three categories:

```
Group 1 → x < a  
Group 2 → a <= x <= b  
Group 3 → x > b
```

We make **three passes** over the original array.

Step 1

Store all elements smaller than `a`.

```
if (x < a)
```

Step 2

Store all elements between `a` and `b`.

```
if (x >= a && x <= b)
```

Step 3

Store all elements greater than `b`.

```
if (x > b)
```

Finally, copy the temporary array back into the original array.

Java Implementation

```
class Solution {
    public void threeWayPartition(int arr[], int a, int b) {

        int n = arr.length;

        int[] temp = new int[n];

        int index = 0;

        // Smaller than a
        for (int x : arr) {
            if (x < a) {
                temp[index++] = x;
            }
        }

        // Between a and b
        for (int x : arr) {
            if (x >= a && x <= b) {
                temp[index++] = x;
            }
        }

        // Greater than b
        for (int x : arr) {
            if (x > b) {
                temp[index++] = x;
            }
        }

        // Copy back
        for (int i = 0; i < n; i++) {
            arr[i] = temp[i];
        }
    }
}
```

Code Explanation

1. Get the array size

```
int n = arr.length;
```

Stores the number of elements in the array.

For example:

```
arr = [1, 4, 3, 2, 5]  
n = 5
```

2. Create a temporary array

```
int[] temp = new int[n];
```

The temporary array will contain the final partitioned result.

Initially:

```
temp = [_, _, _, _, _]
```

3. Create an index

```
int index = 0;
```

`index` tells us where to insert the next element into `temp`.

First Pass — Elements Smaller Than `a`

```
for (int x : arr) {  
    if (x < a) {
```

```
        temp[index++] = x;
    }
}
```

Suppose:

```
arr = [1, 4, 3, 2, 5, 7, 6, 8]
a = 3
b = 6
```

Elements smaller than **3** are:

1, 2

So:

```
temp = [1, 2, _, _, _, _, _, _]
```

index becomes:

```
index = 2
```

Second Pass — Elements Between **a** and **b**

```
for (int x : arr) {
    if (x >= a && x <= b) {
        temp[index++] = x;
    }
}
```

We need:

```
3 <= x <= 6
```

The elements are:

4, 3, 5, 6

Now:

temp = [1, 2, 4, 3, 5, 6, _, _]

index becomes:

index = 6

Third Pass — Elements Greater Than **b**

```
for (int x : arr) {  
    if (x > b) {  
        temp[index++] = x;  
    }  
}
```

We need:

x > 6

The elements are:

7, 8

Now:

temp = [1, 2, 4, 3, 5, 6, 7, 8]

Copy Back to Original Array

```
for (int i = 0; i < n; i++) {  
    arr[i] = temp[i];  
}
```

The original array becomes:

```
arr = [1, 2, 4, 3, 5, 6, 7, 8]
```

Notice that the middle group does **not** have to be sorted.

The important requirement is only:

```
[Elements < a] [Elements between a and b] [Elements > b]
```

Therefore:

```
[1, 2] [4, 3, 5, 6] [7, 8]
```

is also a valid partition.

Dry Run

Consider:

```
arr = [10, 5, 2, 8, 12, 3, 7]  
a = 5  
b = 8
```

First group

Condition:

```
x < 5
```

Elements:

```
2, 3
```

```
temp = [2, 3, _, _, _, _, _]
```

Second group

Condition:

```
5 <= x <= 8
```

Elements:

```
5, 8, 7
```

```
temp = [2, 3, 5, 8, 7, _, _]
```

Third group

Condition:

```
x > 8
```

Elements:

```
10, 12
```

Final:

```
temp = [2, 3, 5, 8, 7, 10, 12]
```

After copying back:

```
arr = [2, 3, 5, 8, 7, 10, 12]
```

This is valid because:


```
2, 3      → < 5
5, 8, 7    → 5 to 8
10, 12     → > 8
```

Why `index++` Is Used

This statement:

```
temp[index++] = x;
```

is equivalent to:

```
temp[index] = x;
index++;
```

For example:

```
index = 0
x = 2
```

After:

```
temp[index++] = x;
```

we get:

```
temp[0] = 2
index = 1
```

The next element will be inserted at index `1`.

Why Three Separate Loops?

The three loops guarantee the required order:

```
First loop
  ↓
Elements < a

Second loop
  ↓
a <= elements <= b

Third loop
  ↓
Elements > b
```

So the final structure is always:

< a	a to b	> b
-----	--------	-----

Complexity Analysis

Let n be the number of elements.

We traverse the array three times:

```
First loop  →  $O(n)$ 
Second loop →  $O(n)$ 
Third loop  →  $O(n)$ 
Copy loop   →  $O(n)$ 
```

Therefore:

Time Complexity = $O(n)$

The temporary array contains n elements:

Space Complexity = $O(n)$

Advantages

- Simple and easy to understand.
- Linear time complexity.
- No sorting is required.
- Works with duplicate values.
- Preserves the relative order within each category.

For example:

```
arr = [4, 2, 4, 1]  
a = 3  
b = 4
```

Result:

```
[2, 1, 4, 4]
```

The order of elements within each group is maintained.

Important Conditions

The three conditions are:

1. Smaller than `a`

```
x < a
```

2. Between `a` and `b`

```
x >= a && x <= b
```

3. Greater than `b`

```
x > b
```

Together, they cover every possible element exactly once.

Alternative: In-Place Three-Way Partition

The temporary-array approach uses $O(n)$ extra space.

A more advanced approach uses **three pointers** and partitions the array in-place:

```
low
mid
high
```

This is similar to the **Dutch National Flag algorithm**.

Its complexity is:

```
Time   :  $O(n)$ 
Space  :  $O(1)$ 
```

However, the three-pass temporary-array solution is easier to understand and implement.

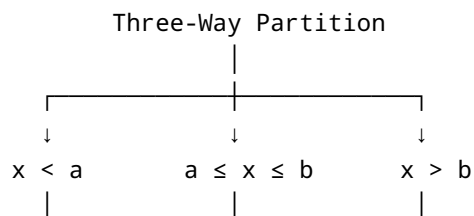
Key DSA Pattern

This problem teaches the **partitioning technique**.

Whenever an array needs to be divided into categories based on conditions, think about:

1. Define the groups.
2. Process elements according to their conditions.
3. Place them in their required region.

For this problem:



↓
First

↓
Second

↓
Last

Final Complexity

Time Complexity : $O(n)$

Space Complexity : $O(n)$

Key Takeaway

Partition the array into three categories:

< a | a to b | > b

Your solution achieves **$O(n)$ time** by making a constant number of passes through the array.